

An Agentic Intelligent Code Refactoring Assistant for Legacy Systems

R26-SE-008

Developer Interaction & Workflow Orchestration Agent

Project Proposal Report

Bandara S M Y M

IT22277886

B.Sc. (Hons) Degree in Information Technology Specialized in
Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
IT22277886	Bandara S M Y M	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Dr. Kalpani Manathunga	17/03/26	
Mr. Jeewaka Perera	13-03-2026	

ABSTRACT

Modern software systems continuously evolve, leading to the accumulation of technical debt and code quality issues. Developers often rely on static analysis tools, integrated development environments (IDEs), and automated refactoring utilities to maintain code quality. However, many existing tools either require significant manual intervention from developers or perform automated changes with limited transparency and developer control. This creates a trust gap between developers and automated refactoring systems, particularly when large-scale code transformations are involved.

Recent advancements in AI-powered software engineering have introduced intelligent agents capable of detecting code issues and suggesting refactoring strategies. Despite these advances, current solutions lack a coordinated mechanism that effectively integrates multiple refactoring agents while maintaining meaningful developer interaction. Most existing tools operate in isolated workflows without a centralized orchestration layer that manages the interaction between automated agents and developers.

This research proposes the development of a **Developer Interaction & Workflow Orchestration Agent**, which acts as the central coordination component within an Intelligent Agentic Code Refactoring System. The proposed agent will manage the workflow between automated analysis agents, refactoring planning agents, and code transformation agents while ensuring that developers remain actively involved in the decision-making process through a human-in-the-loop approach. The system will provide an interactive interface, potentially implemented as a Visual Studio Code extension or web dashboard, allowing developers to visualize detected issues, review suggested refactoring plans, approve or reject changes, and monitor the results of automated transformations.

The implementation will utilize modern technologies such as **large language models, microservice-based architectures, workflow orchestration frameworks, and developer-centric interfaces integrated with version control systems**. The expected outcomes include improved transparency in automated refactoring, increased developer trust in AI-assisted tools, and more efficient software maintenance workflows. The system has potential commercial value for software development teams, DevOps environments, and enterprise software maintenance platforms.

ACKNOWLEDGEMENT

I would like to express my sincere appreciation to my research supervisor for their continuous support, guidance, and valuable advice throughout the development of this research proposal. Their expertise, direction, and constructive feedback played a vital role in improving the quality and clarity of my work.

I am also grateful to my co-supervisor for their encouragement, insightful suggestions, and technical assistance, which significantly contributed to strengthening this proposal.

In addition, I would like to thank the academic staff of the Faculty of Computing for providing the necessary knowledge, resources, and a supportive academic environment that enabled me to successfully carry out this research.

Finally, I extend my heartfelt thanks to everyone who provided support, guidance, and encouragement, both directly and indirectly, in the completion of this proposal.

Table of Contents

ABSTRACT	3
ACKNOWLEDGEMENT	4
LIST OF TABLES	7
LIST OF FIGURES	8
LIST OF ABBREVIATIONS	9
LIST OF APPENDICES	9
1.INTRODUCTION	10
1.1 Background and Context.....	10
1.2 Literature Review	11
1.3 Research Gap	13
2.OBJECTIVE	14
2.1 Main Objective.....	14
2.2 Specific Objectives	14
3.METHODOLOGY	16
3.1 Research Approach.....	16
3.2 Technical Methods and Algorithms	16
3.3 Tools and Platforms with Justification	18
3.4 Data Collection and Ethical Considerations	18
3.5 Validation Strategy and Performance Metrics	19
4.HIGH-LEVEL SYSTEM ARCHITECTURE	20
4.1 Overview of the Proposed Solution and Its Components.....	20
4.2 System Diagram of Components.....	22
4.3 Highlighting the Individual Component and Its Parts	22
4.4 Component Interactions	23
4.5 Data Flow Explanation	24
4.6 Scalability and Security Considerations	26
5.USER REQUIREMENTS	27
5.1 Stakeholder Identification.....	27
5.2 Functional Requirements	27
5.3 Non-Functional Requirements.....	28
6.COMMERCIALIZATION PLAN	29

7.BUDGET AND JUSTIFICATION	31
8.WORK BREAKDOWN STRUCTURE (WBS).....	33
9.GANTT CHART	35
10.REFERENCES LIST	36
11.APPENDICES.....	37

LIST OF TABLES

Table 1. Literature review comparison.....	12
Table 2. Tools and platforms.....	18
Table 5 Estimated Budget for System Implementation and Experimentation.....	31
Table 6 Task Distribution Among Team Members	33
Table 7Project Timeline Alignment.....	34
Table 8- Gantt Chart	35

LIST OF FIGURES

Figure 1. System Diagram	22
Figure 2 Data Flow Diagram	25

LIST OF ABBREVIATIONS

API – Application Programming Interface

AST – Abstract Syntax Tree

CI/CD – Continuous Integration / Continuous Deployment

IDE – Integrated Development Environment

LLM – Large Language Model

MAS – Multi-Agent System

REST – Representational State Transfer

SaaS – Software as a Service

SDG – Sustainable Development Goal

SE – Software Engineering

SST – Software Systems & Technologies

UI – User Interface

UX – User Experience

VS Code – Visual Studio Code

WBS – Work Breakdown Structure

LIST OF APPENDICES

Appendix A – Detailed System Architecture Diagram

Appendix B – Data Flow Diagram of the Developer Interaction & Workflow Orchestration Agent

Appendix D – Preliminary User Interface Mockups for the Developer Dashboard

Appendix F – Sample Refactoring Plan Output Generated by the Planning Agent

Appendix I – Project Timeline and Detailed Gantt Chart

1. INTRODUCTION

1.1 Background and Context

Legacy software systems are widely used across industries, but they often suffer from accumulated technical debt, outdated design patterns, and increasing complexity. As systems evolve over time, the maintainability and readability of code degrade, making it difficult for developers to understand and modify existing implementations.

Developers working with legacy codebases frequently face challenges such as unclear dependencies, hidden design flaws, and undocumented logic. Research studies indicate that developers spend **58%–70% of their development time understanding existing source code rather than writing new functionality**, demonstrating the significant effort required for software maintenance.[1]

Furthermore, poor code quality and technical debt can waste **up to 42% of developers' working time**, leading to increased maintenance costs and reduced productivity in software development organizations. [2]

Although several tools exist to detect code smells or suggest simple refactorings, they often lack intelligent coordination and developer-centric interaction mechanisms. Static analysis tools generate reports, but developers must manually interpret them and decide which refactoring steps to perform.

In an intelligent multi-agent refactoring system, different agents handle tasks such as code analysis, refactoring planning, and transformation. However, without a centralized orchestration layer, these agents cannot effectively collaborate with developers.

The **Developer Interaction & Workflow Orchestration Agent** addresses this gap by acting as the bridge between automated refactoring agents and human developers. It coordinates the workflow of multiple agents while providing developers with a transparent interface to review, control, and refine refactoring actions.

This research aligns with **Sustainable Development Goal (SDG) 9 – Industry, Innovation, and Infrastructure** by improving software infrastructure through intelligent automation and improved developer productivity.

This research aligns with the **Software Systems & Technologies (SST) research cluster**, which focuses on the design, development, and improvement of advanced software systems and technological solutions that enhance software development processes and system reliability. The proposed **Developer Interaction & Workflow Orchestration Agent** contributes to this cluster by introducing an intelligent coordination mechanism that integrates automated refactoring agents with developer-driven decision-making. By enabling structured workflows, human-in-the-loop interaction, and transparent collaboration between multiple intelligent agents, the proposed solution improves software maintenance practices and supports the development of more reliable and maintainable software systems, particularly in the context of legacy software modernization.

1.2 Literature Review

Software refactoring has been widely studied as an essential technique for improving code quality, maintainability, and system evolution in large software systems. Traditional research in software refactoring mainly focuses on static code analysis and rule-based detection of code smells. Tools such as SonarQube and JDeodorant assist developers in identifying potential quality issues such as duplicated code, large classes, or high coupling. However, these tools primarily provide analytical reports and recommendations rather than executing coordinated refactoring workflows. As a result, developers must manually interpret these results and decide how to apply refactoring actions, which becomes challenging when working with complex legacy systems.

Modern integrated development environments (IDEs) such as IntelliJ IDEA and Visual Studio Code provide built-in refactoring operations including rename, extract method, move class, and inline refactoring. These features allow developers to safely perform small-scale structural modifications within the codebase. Although IDE-based refactoring tools provide reliable and safe transformations, they are limited to developer-initiated actions and typically operate only within a single file or module. They lack system-wide reasoning, automated planning, and intelligent coordination across multiple files or components. Consequently, developers must still manually analyze the architecture and determine the appropriate sequence of refactoring actions.

Recent advancements in artificial intelligence have introduced AI-powered development assistants such as GitHub Copilot, Cursor AI, Zencoder, and Qodo. These systems utilize large language models to understand programming context and automatically generate code suggestions, including refactoring recommendations. Some modern tools can analyze entire repositories and perform coordinated changes across multiple files. For example, AI-driven refactoring assistants can modernize legacy code structures, remove redundant code, and improve architectural design. These approaches demonstrate significant potential for automating complex refactoring tasks.

Despite these advancements, several limitations remain in existing AI-based refactoring tools. Many systems operate as “black-box” models that generate code modifications without providing clear reasoning or verification mechanisms. This lack of transparency reduces developer trust, especially when modifying legacy systems where errors may introduce critical failures. Additionally, most current AI tools focus primarily on productivity during active development rather than addressing the complex challenges associated with maintaining and modernizing legacy systems.

Another key limitation in the current research landscape is the absence of a coordinated orchestration mechanism that manages the collaboration between different refactoring components and human developers. While some systems can detect code smells or suggest improvements, they typically operate independently without integrating analysis, planning, transformation, and validation stages into a unified workflow. As a result, developers still need to manually manage the refactoring process and coordinate multiple tools.

Current automated refactoring tools often operate in a **black-box manner**, where the system analyzes the code and directly applies refactoring changes without clearly showing the intermediate decision process to developers. Developers typically only see the **final refactored output after the transformation is completed**, without understanding which code smells were detected, what refactoring strategies were selected, or how the system executed the modifications. Due to this **lack of transparency, explanation, and developer involvement**, developers often find it difficult to trust automated refactoring decisions and verify whether the applied changes are safe and appropriate for the system.

To address these limitations, recent research trends have explored the concept of **agent-based software engineering**, where specialized agents perform tasks such as program analysis, refactoring planning, transformation, and validation. Within such systems, the **Developer Interaction & Workflow Orchestration Agent** plays a critical role in coordinating these agents and maintaining human oversight. This agent provides an interactive interface, typically through a dashboard or IDE extension, allowing developers to review detected issues, inspect refactoring plans, approve or reject transformations, and monitor the results of automated changes. By enabling a human-in-the-loop workflow, the orchestration agent ensures transparency, improves developer trust, and enables safe collaboration between automated systems and human developers.

Therefore, the proposed **Developer Interaction & Workflow Orchestration Agent** extends beyond existing static analysis tools and AI coding assistants by providing a centralized coordination layer that integrates automated agents with developer decision-making. This approach improves workflow transparency, enables structured collaboration between agents, and supports safe, large-scale refactoring of legacy software systems.

Comparison of Existing Systems vs Proposed Orchestration Agent

Table 1. Literature review comparison

Feature / Capability	Static Analysis Tools (SonarQube)	IDE Refactoring Tools (IntelliJ / VS Code)	AI Coding Assistants (Copilot / Cursor / Qodo)	Proposed Developer Interaction & Workflow Orchestration Agent
Code smell detection	Yes	Limited	Yes	Yes
Automated refactoring execution	No	Limited	Partial	Yes

Multi-file / repository level refactoring	Limited	Mostly local	Yes	Yes
Intelligent refactoring planning	No	No	Partial	Yes
Human-in-the-loop control	No	Manual only	Limited	Full support
Explainable decision making	No	No	Limited	Yes
Workflow orchestration between agents	No	No	No	Yes
Refactoring validation and rollback	No	Partial	Limited	Yes
Developer dashboard / visualization	Yes	No	Partial	Yes
End-to-end refactoring pipeline	No	No	Partial	Yes

1.3 Research Gap

Current refactoring systems already support local IDE-based refactoring, repository-wide AI-assisted code changes, and static-analysis-based issue detection. However, they still do not provide a **dedicated developer interaction and workflow orchestration layer** that coordinates the full refactoring lifecycle while keeping the developer continuously in control. The literature shows that IDE tools are mostly developer-triggered and local, AI agents are powerful but often act as semi-autonomous or black-box systems, and static analysis platforms mainly detect and report problems rather than manage end-to-end action.

Existing refactoring tools and AI-based coding assistants support detection, suggestion, and even large-scale automated transformation, but they do not sufficiently address **developer-centered workflow orchestration**. There is still no integrated mechanism that proactively coordinates refactoring activities across agents while allowing developers to review decisions, control execution, provide feedback, track changes, and maintain trust through transparency and auditability. Therefore, a **Developer Interaction & Workflow Orchestration Agent** is needed to bridge the gap between passive reporting tools, reactive coding assistants, and highly autonomous refactoring agents.

2. OBJECTIVE

2.1 Main Objective

To design and develop a **Developer Interaction & Workflow Orchestration Agent** that coordinates the refactoring workflow between multiple agents and developers, while enabling effective review, approval, feedback, and control over automated refactoring decisions.

2.2 Specific Objectives

- To design an interactive dashboard or IDE extension that allows developers to provide repository inputs, configure refactoring preferences, and monitor the progress of the refactoring workflow.
- To coordinate communication and task flow between the Code Understanding Agent, Refactoring Planning Agent, and Transformation Agent within the automated refactoring process.
- To enable developers to review, approve, reject, reprioritize, or modify refactoring plans generated by the Refactoring Planning Agent before code transformation begins.
- To enable developers to inspect transformed code, validation results, and rollback options after the Transformation Agent applies refactoring changes.
- To provide human-in-the-loop control mechanisms that allow developers to approve, reject, modify, or roll back automated refactoring actions.
- To maintain audit logs and historical records of refactoring plans, developer decisions, applied changes, validation outcomes, and rollback actions for traceability.
- To visualize before-and-after code quality improvements so developers can evaluate the impact of applied refactoring actions.
- To incorporate developer feedback into the orchestration workflow to improve future refactoring recommendations and interactions.

3. METHODOLOGY

3.1 Research Approach

This research adopts the **Design Science Research (DSR) methodology**, as the primary objective is to design, develop, and evaluate a software artifact that addresses a real-world problem in automated code refactoring. The artifact developed in this study is the **Developer Interaction & Workflow Orchestration Agent**, which enables a **human-in-the-loop multi-agent refactoring workflow**.

DSR is appropriate because the study focuses on:

- building a **practical solution** rather than only theoretical analysis,
- integrating **developer decision-making into automated workflows**, and
- evaluating the artifact through **real-world software repositories and usability metrics**.

The research follows DSR phases: problem identification, objective definition, artifact design, implementation, demonstration, and evaluation.

3.2 Technical Methods and Algorithms

The system is designed as an **approval-driven orchestration workflow** that coordinates multiple intelligent agents. The Developer Interaction & Workflow Orchestration Agent uses the following technical methods:

1. Workflow Orchestration Algorithm

- Implements a **state-based workflow model**
- Each stage (analysis → planning → approval → transformation → validation) is treated as a state
- Transitions occur based on **developer decisions (approve/reject/modify)**

Key logic:

- If *approved* → proceed to next stage
 - If *modified* → re-trigger planning agent
 - If *rejected* → terminate or rollback
-

2. Multi-Agent Coordination Mechanism

The orchestration agent coordinates:

- Code Understanding Agent → detects code smells
- Planning Agent → generates refactoring plan
- Transformation Agent → applies changes

- Validation Agent → verifies correctness

Communication is handled through:

- structured JSON outputs
 - REST APIs between services
-

3. Approval-Gated Execution Model (SO1)

- Refactoring actions are executed **only after developer approval**
 - Ensures **human control and trust**
-

4. Version Control & Rollback Algorithm (SO3)

- Uses Git branching strategy:
 - Create snapshot before refactoring
 - Apply changes in isolated branch
 - Rollback triggered by:
 - validation failure
 - developer rejection
-

5. Audit Logging Mechanism (SO5)

- Immutable logging of:
 - workflow states
 - agent outputs
 - developer decisions
 - Stored in PostgreSQL for traceability
-

6. Visualization Data Processing (SO4)

- Compute before–after metrics:
 - code complexity
 - duplication
 - maintainability indicators
 - Send structured data to frontend for visualization
-

3.3 Tools and Platforms with Justification

Table 2. Tools and platforms

Component	Technology	Justification
Backend	Python + Flask	Lightweight, flexible API development and easy integration with AI/agent-based systems
Workflow Orchestration	LangGraph + Prefect	Supports multi-agent coordination, conditional branching, and approval-based workflows
Frontend	React.js + Tailwind CSS	Enables interactive dashboard for visualization, approvals, and rollback controls
Database	PostgreSQL	Reliable storage for audit logs and workflow states with strong consistency
Version Control Integration	GitHub API / GitPython	Enables repository access, branching, commit tracking, and safe rollback
Agent Communication	REST APIs + JSON	Standardized and scalable communication between system components

These tools are selected to ensure **scalability, transparency, and maintainability** of the system.

3.4 Data Collection and Ethical Considerations

Data Collection

Data will be collected from the following sources:

- 1. **Public GitHub repositories**
 - Used for testing refactoring workflows
 - Includes legacy and maintainability-challenged projects
- 2. **System-generated data**
 - workflow logs
 - refactoring plans
 - validation results
 - rollback events
- 3. **Developer interaction data**
 - approval/rejection decisions
 - feedback on refactoring plans
 - usability responses

Ethical Considerations

- Only **publicly available repositories** will be used
- No sensitive or private codebases will be accessed
- Developer participation (if applicable) will be:
 - voluntary
 - anonymized
- System logs will not store personal identifiers

- Data will be used strictly for **research purposes**
-

3.5 Validation Strategy and Performance Metrics

1. Technical Validation

The system will be evaluated based on:

- **Workflow correctness**
 - successful execution of approval-gated stages
 - **Agent coordination efficiency**
 - smooth transition between agents
 - **Rollback reliability**
 - ability to revert changes without data loss
 - **Validation success rate**
 - percentage of refactored code passing validation checks
-

2. Code Quality Improvement Metrics

To evaluate refactoring effectiveness:

- Cyclomatic complexity reduction
 - Code duplication reduction
 - Maintainability index improvement
 - Number of resolved code smells
-

3. Usability Evaluation

Usability will be measured using:

- **System Usability Scale (SUS)** or Likert-scale survey
 - Metrics include:
 - ease of use of dashboard
 - clarity of workflow stages
 - trust in automated recommendations
 - satisfaction with rollback and control features
-

4. Performance Metrics

- Execution time per workflow stage
- Response time for developer interactions
- System scalability with larger repositories

4. HIGH-LEVEL SYSTEM ARCHITECTURE

The proposed architecture consists of four primary agents:

1. Code Understanding Agent
2. Refactoring Planning Agent
3. Code Transformation Agent
4. Developer Interaction & Workflow Orchestration Agent

The **Developer Interaction Agent** acts as the central interface connecting the automated agents with the developer.

Input to the system includes a GitHub repository or local project workspace. The Code Understanding Agent analyzes the code and generates quality reports. The Planning Agent produces refactoring strategies, and the Transformation Agent executes safe code modifications.

The Workflow Orchestration Agent collects results from all agents and presents them to the developer through an interactive dashboard or IDE extension. Developers can review proposed changes, approve or reject actions, and provide feedback that influences subsequent agent decisions.

The architecture ensures transparency, traceability, and collaboration between human developers and automated agents.

4.1 Overview of the Proposed Solution and Its Components

The proposed solution is a **human-in-the-loop automated refactoring support system** in which the **Developer Interaction & Workflow Orchestration Agent** serves as the central coordinating component. The purpose of this architecture is to ensure that automated refactoring activities are not performed in isolation, but rather in a controlled workflow where developers can review, approve, reject, or modify important decisions at key stages.

The system begins when the developer provides a **GitHub repository link**, together with **user-defined refactoring preferences** and optional **code complexity thresholds**. Based on this input, the system executes a sequence of analysis, planning, transformation, validation, and review activities. While specialized agents perform code understanding, refactoring planning, and transformation, the Developer Interaction & Workflow Orchestration Agent manages the interactions among these components and ensures that the workflow proceeds in an organized, transparent, and traceable manner.

The overall architecture consists of the following main components:

Developer Interface

The Developer Interface is implemented as a **React-based dashboard or IDE extension**. It acts as the primary interaction layer through which the developer submits repository

information, configures refactoring preferences, reviews generated plans, inspects transformed code, and makes approval or rollback decisions.

Developer Interaction & Workflow Orchestration Agent

This is the main component of the proposed research. It acts as the central controller of the system by coordinating communication between the developer and the specialized refactoring agents. It is also responsible for workflow progression, approval handling, developer feedback management, rollback coordination, and maintenance of traceable system records.

Code Understanding Agent

The Code Understanding Agent is responsible for analyzing the source code and generating structural and quality-related outputs, code complexity assessments, and quality reports.

Refactoring Planning Agent

The Refactoring Planning Agent uses the outputs of the Code Understanding Agent to generate prioritized refactoring strategies and planning recommendations for developer review.

Transformation Agent

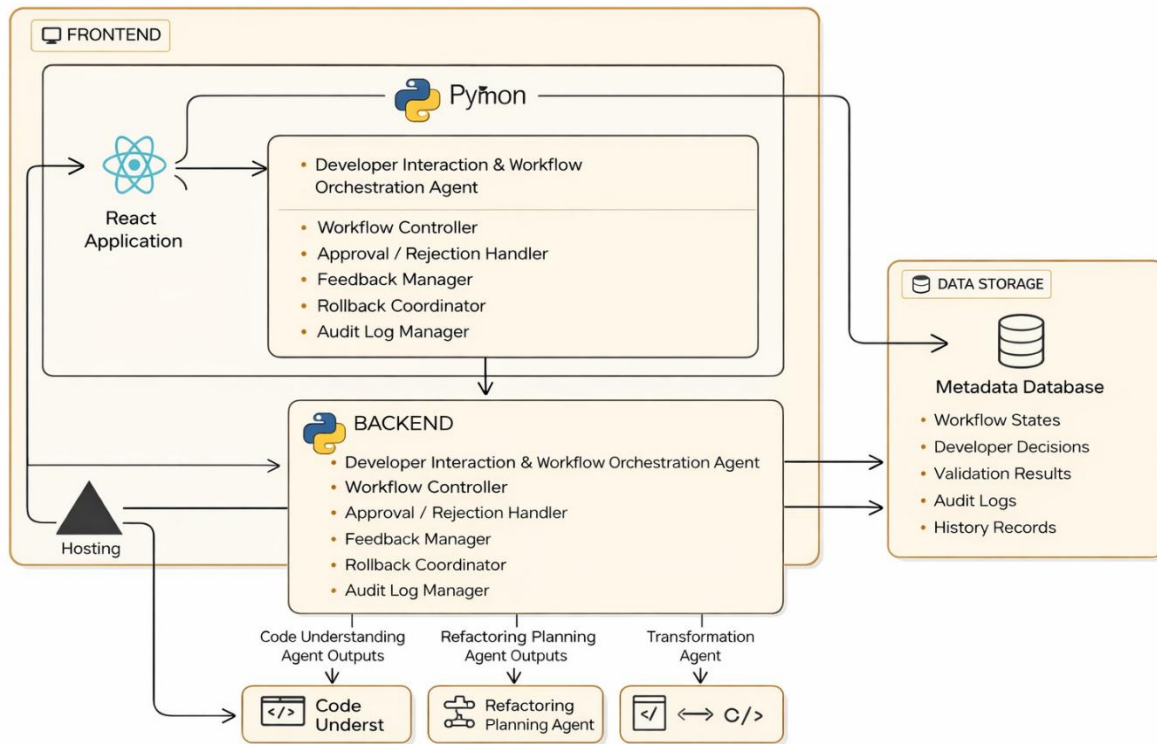
The Transformation Agent applies the approved refactoring actions to the source code, performs validation procedures, and supports rollback when transformation outcomes are unsatisfactory or invalid.

Metadata and Logs Database

The Metadata and Logs Database stores workflow states, refactoring plans, developer decisions, validation outputs, rollback records, and audit trails. This repository ensures traceability, accountability, and historical analysis of the refactoring process.

4.2 System Diagram of Components

Table 3. System Diagram



The above diagram represents the high-level system architecture of the proposed solution:

4.3 Highlighting the Individual Component and Its Parts

The **Developer Interaction & Workflow Orchestration Agent** is the principal contribution of this study. Unlike the other agents, which focus on analysis, planning, or code modification, this component is designed to manage the interaction and coordination layer of the refactoring system. Its primary function is to ensure that the developer remains actively involved in the workflow and that the outputs of the specialized agents are delivered, reviewed, and acted upon in a structured manner.

The internal structure of the Developer Interaction & Workflow Orchestration Agent consists of the following sub-components:

Workflow Controller

The Workflow Controller is responsible for managing the sequence of refactoring activities. It tracks the current workflow state and ensures that the system progresses through stages such as input submission, code analysis, planning, developer review, code transformation,

validation, and final acceptance or rollback. This ensures that the overall workflow follows a consistent and logically ordered process.

Approval / Rejection Handler

This sub-component manages developer decisions during key review stages. It enables the developer to approve, reject, modify, or reprioritize the refactoring strategies generated by the Refactoring Planning Agent. It also supports decision-making after code transformation by allowing the developer to inspect the transformed output and determine whether the changes should be accepted or rolled back.

Feedback Manager

The Feedback Manager captures developer comments, preferences, and corrective actions during the workflow. This feedback can be incorporated into future orchestration decisions, thereby improving alignment between automated refactoring recommendations and project-specific requirements.

Audit Log Manager

The Audit Log Manager records all important system activities, including generated plans, developer approvals, transformation results, validation outcomes, and rollback operations. This sub-component is essential for maintaining traceability and accountability throughout the refactoring process.

Rollback Coordinator

The Rollback Coordinator works in conjunction with the Transformation Agent to restore the previous version of the source code when a developer rejects the applied changes or when validation procedures indicate that the refactoring outcome is not acceptable.

Collectively, these sub-components establish the Developer Interaction & Workflow Orchestration Agent as the central supervisory mechanism of the proposed architecture.

4.4 Component Interactions

The proposed architecture operates through structured interactions among the developer, the interface layer, the orchestration agent, and the specialized refactoring agents.

Initially, the developer interacts with the system through the **React-based dashboard or IDE extension**. At this stage, the developer submits the repository or workspace, specifies refactoring preferences, and optionally defines complexity thresholds. These inputs are then forwarded to the **Developer Interaction & Workflow Orchestration Agent**, which initializes and controls the refactoring workflow.

The orchestration agent first coordinates with the **Refactoring Planning Agent**, which generates prioritized refactoring strategies based on the identified issues finalized by the **Code Understanding Agent**.

Once the planning stage is completed, the proposed strategies are returned to the **Developer Interaction & Workflow Orchestration Agent**, which presents them to the developer for

review through the interface. At this point, the developer can approve, reject, reprioritize, or modify the suggested plan. Only the approved refactoring actions are forwarded to the **Transformation Agent**.

The **Transformation Agent** then applies the approved changes to the codebase and performs validation checks to ensure correctness and consistency. The transformed outputs and validation results are returned to the **orchestration agent**, which presents them to the developer for post-transformation review. Based on this review, the developer may accept the changes or request a rollback.

Throughout all of these interactions, the **Metadata and Logs Database** records workflow events, decisions, outputs, and history. Therefore, the orchestration agent functions as the communication bridge and control mechanism that links all system components together.

4.5 Data Flow Explanation

The data flow of the **Developer Interaction & Workflow Orchestration Agent** is centered on managing developer interaction and workflow control after refactoring-related outputs have been generated by other agents. In this architecture, the component becomes active once the **Code Understanding Agent** provides the analysis outputs and the **Refactoring Planning Agent** generates the proposed refactoring plan. The following stages describe the data flow of this component.

1. **Code Understanding Output Transfer**

The data flow begins when the **Code Understanding Agent** produces analysis outputs related to the legacy source code. These outputs may include code quality findings, structural summaries, complexity-related information, and identified refactoring opportunities. This information serves as the input for the next stage of refactoring planning.

2. **Refactoring Plan Generation**

The analysis outputs are processed by the **Refactoring Planning Agent** to generate candidate refactoring actions and prioritized recommendations. The generated refactoring plan represents the first major data artifact that is handled by the **Developer Interaction & Workflow Orchestration Agent**.

3. **Developer Review and Approval**

Once the refactoring plan is generated, it is forwarded to the **Developer Interaction & Workflow Orchestration Agent**, which presents it to the developer through the dashboard or IDE extension. At this stage, the developer reviews the proposed refactoring actions and may approve, reject, modify, or reprioritize them. The developer's decisions are captured as workflow control data for the subsequent stage.

4. **Forwarding Approved Actions for Transformation**

After the review stage, the orchestration agent forwards only the approved or revised refactoring actions to the **Transformation Agent**. This ensures that no automated transformation is applied without developer involvement and approval.

5. **Transformation and Validation**

The **Transformation Agent** applies the selected refactoring changes to the source code and performs the required validation procedures. These validation activities

ensure that the transformed code remains correct, consistent, and suitable for further use. The output of this stage includes transformed code results, validation outcomes, and rollback-related information.

6. **Developer Review of Results**

The transformation and validation outputs are returned to the **Developer Interaction & Workflow Orchestration Agent**, which presents them to the developer for a second review stage. At this point, the developer examines the applied changes and validation results in order to assess whether the refactored code is acceptable.

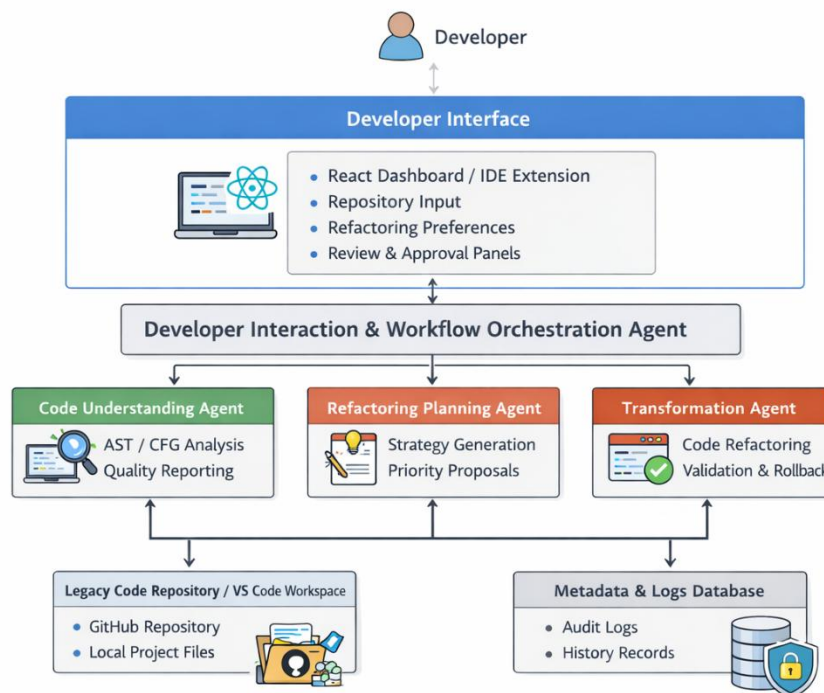
7. **Acceptance or Rollback Decision**

Based on the post-transformation review, the developer makes a final decision. If the transformed code is satisfactory, the changes are accepted. If the results are not satisfactory, or if validation indicates problems, the developer may request rollback. This stage ensures that the developer retains final control over the outcome of the refactoring process.

8. **Logging and History Storage**

Throughout all stages of the workflow, the **Developer Interaction & Workflow Orchestration Agent** records the relevant workflow events and decision data in the system's logging and history storage layer. These records may include generated plans, approval decisions, modifications, validation results, accepted changes, rollback actions, and workflow state transitions. This supports traceability, accountability, and future analysis of the refactoring workflow.

Table 4 Data Flow Diagram



4.6 Scalability and Security Considerations

Scalability Considerations

The proposed architecture is designed according to a modular, component-based structure, which supports future system expansion and maintainability. Since each major function is separated into an independent agent or subsystem, the architecture can be scaled more effectively than a monolithic design.

From a scalability perspective, the following considerations are relevant:

- The separation of analysis, planning, transformation, and orchestration functions allows each component to be improved or extended independently.
- The architecture can be adapted to support multiple repositories, multiple workflow sessions, or concurrent users in future implementations.
- The React-based interface can be extended with additional visualization panels, filters, and control mechanisms without affecting the core orchestration logic.
- The Python backend can be enhanced to support larger-scale projects by incorporating asynchronous processing, distributed task execution, or container-based deployment.
- Additional specialized agents may be integrated into the architecture as future enhancements without redesigning the entire system.

These characteristics demonstrate that the architecture has been designed with extensibility and practical deployment considerations in mind.

Security Considerations

Because the proposed system handles source code, repository information, and automated code transformations, security is an essential architectural concern. The system must ensure that sensitive development assets are protected and that automated actions do not introduce uncontrolled risks.

The following security considerations are therefore important:

- **Access Control:** Only authorized users should be able to upload repositories, access workflow dashboards, and approve or reject refactoring actions.
- **Secure Repository Handling:** Repository links, access tokens, and workspace information must be securely managed to prevent unauthorized access.
- **Input Validation:** Repository paths, URLs, preference settings, and workflow requests should be validated before processing to avoid invalid or malicious input.
- **Safe Change Management:** Refactoring actions should not be finalized without explicit developer review and approval at critical stages.
- **Rollback Protection:** The system must preserve previous code states and support reliable rollback to prevent permanent damage from incorrect transformations.
- **Auditability:** All actions, including automated agent outputs and developer decisions, should be logged to provide accountability and traceability.
- **Secure Communication:** Communication between the React frontend and the Python backend should use secure channels and authenticated API access.
- **Least-Privilege Execution:** Transformation processes should operate with restricted permissions to minimize the impact of errors or misuse.

These security measures are necessary to ensure that the proposed architecture remains reliable, accountable, and suitable for integration into practical software development environments.

5. USER REQUIREMENTS

5.1 Stakeholder Identification

The proposed **Developer Interaction & Workflow Orchestration Agent** involves several stakeholders who interact with or benefit from the system in different ways.

1. Developers

Developers are the primary users of the system. They interact with the dashboard or IDE extension to review refactoring plans, approve or reject suggested actions, inspect transformation results, and request rollback when necessary.

2. Software Maintainers

Software maintainers are responsible for improving and sustaining legacy systems over time. They use the system to monitor refactoring activities, review historical changes, and ensure that code improvements align with maintenance goals.

3. Development Teams / Project Teams

Development teams benefit from the system through improved coordination, visibility, and traceability of refactoring workflows. Team members can use the logs and workflow history to understand past decisions and current refactoring status.

4. System Administrators

System administrators are responsible for managing access control, maintaining the deployment environment, and ensuring that the platform operates securely and reliably.

5. Researchers / Evaluators

Researchers or evaluators may use the system to assess workflow efficiency, developer interaction quality, usability, and the effectiveness of the orchestration process.

5.2 Functional Requirements

The functional requirements describe what the **Developer Interaction & Workflow Orchestration Agent** must do in order to support developer-centered workflow management.

FR1. The system shall present refactoring plans generated by the **Refactoring Planning Agent** to the developer for review.

FR2. The system shall allow the developer to approve, reject, modify, or reprioritize proposed refactoring actions before transformation begins.

- FR3.** The system shall forward only the approved or revised refactoring actions to the **Transformation Agent**.
- FR4.** The system shall present transformed code outputs and validation results to the developer after the transformation stage is completed.
- FR5.** The system shall allow the developer to accept the transformed results or request rollback after reviewing the applied changes.
- FR6.** The system shall coordinate workflow communication between the **Refactoring Planning Agent**, **Transformation Agent**, and developer interface.
- FR7.** The system shall track and manage workflow states such as plan generated, review pending, approved, transformed, validated, accepted, and rolled back.
- FR8.** The system shall record developer decisions, including approvals, rejections, modifications, and rollback requests.
- FR9.** The system shall maintain audit logs and history records of workflow events, decisions, and validation outcomes.
- FR10.** The system shall display before-and-after summaries of refactoring outcomes and validation results to support developer evaluation.
- FR11.** The system shall capture developer feedback during the review stages for future workflow improvement.
- FR12.** The system shall notify or update the developer when workflow stages change, such as when a plan is ready for review or when validation results become available.
-

5.3 Non-Functional Requirements

The non-functional requirements describe the quality attributes that the **Developer Interaction & Workflow Orchestration Agent** must satisfy.

Security Requirements

- NFR1.** The system shall ensure that only authorized users can access the dashboard, review refactoring plans, and make approval or rollback decisions.
- NFR2.** The system shall securely manage workflow data, validation results, and history records to prevent unauthorized access or tampering.
- NFR3.** The system shall maintain auditability by recording all major workflow events and developer actions.
- NFR4.** The system shall support safe rollback mechanisms to prevent irreversible code changes.

Performance Requirements

NFR5. The system shall present refactoring plans and transformation results to the developer without unnecessary delay.

NFR6. The system shall support efficient workflow coordination between components so that user interactions occur smoothly.

NFR7. The system shall handle multiple workflow events and decision records without significant performance degradation.

Usability Requirements

NFR8. The system shall provide a simple and understandable interface for reviewing refactoring plans and transformation results.

NFR9. The system shall display workflow states, validation outcomes, and rollback options clearly to support developer decision-making.

NFR10. The system shall minimize developer effort by providing clear controls for approval, rejection, modification, and rollback.

NFR11. The system shall present before-and-after results in a way that is easy for developers to interpret.

Reliability Requirements

NFR12. The system shall reliably preserve workflow states and developer decisions throughout the refactoring process.

NFR13. The system shall ensure that no transformation action proceeds without the required developer approval.

NFR14. The system shall consistently store logs and historical records for future reference.

Scalability Requirements

NFR15. The system shall support extension to larger projects, additional workflow sessions, and future integration with other agents or tools.

NFR16. The system shall maintain acceptable performance as the number of refactoring plans, workflow records, and developer interactions increases.

6. COMMERCIALIZATION PLAN

The proposed Automated Multi-Agent Software Refactoring System has strong commercialization potential within the global software development and software maintenance industry. The primary target market includes software development companies, technology startups, and large enterprises that maintain complex software systems or legacy

codebases. These organizations frequently struggle with technical debt, poor code quality, and the increasing complexity of maintaining large software systems over time. Development teams that actively use AI-assisted coding tools such as GitHub Copilot or other large language model–based coding assistants represent a particularly relevant customer segment, as these tools often accelerate code generation without ensuring long-term architectural quality. The main users of the system would include software developers, software architects, DevOps engineers, and quality assurance teams responsible for maintaining and improving code quality.

The value proposition of the proposed system lies in its ability to provide an automated and intelligent refactoring workflow that analyzes code quality issues and generates structured refactoring plans before performing code transformations. Unlike traditional static analysis tools that only detect code smells or provide isolated recommendations, the proposed multi-agent framework integrates code analysis, strategic refactoring planning, safe code transformation, and validation into a unified pipeline. The Refactoring Decision and Planning Agent (RDP Agent) plays a key role in this architecture by generating dependency-aware refactoring plans that guide the transformation process and ensure that refactoring actions are applied in an optimal sequence.

From a commercialization perspective, the system can be delivered as a developer productivity platform integrated into modern software development environments. The system could be deployed as an integrated development environment (IDE) plugin, a cloud-based code quality analysis service, or an enterprise software maintenance platform integrated with version control systems such as GitHub or GitLab. A Software-as-a-Service (SaaS) revenue model could be adopted, where development teams subscribe to the platform on a monthly or annual basis to analyze repositories, generate refactoring plans, and improve software quality across projects.

The operational costs associated with commercial deployment would primarily include cloud infrastructure for code analysis and machine learning processing, platform maintenance and updates, and technical support services for enterprise users. A tiered pricing strategy could be implemented to accommodate different types of customers. For example, smaller development teams and startups could access a basic version of the platform with limited analysis capabilities, while larger enterprises could purchase premium plans that support large-scale repository analysis, advanced refactoring planning, integration with DevOps pipelines, and team collaboration features.

The system maintains a competitive advantage by providing a planning-driven automated refactoring solution that considers dependencies, architectural constraints, and expected quality improvements before executing transformations. While many existing tools focus only on detecting code smells or performing individual refactoring actions, the proposed framework introduces an intelligent planning layer that coordinates multiple refactoring operations within a structured workflow. This approach enables safer and more effective large-scale refactoring of complex software systems.

In terms of intellectual property, the core innovation of the system lies in the multi-agent architecture and the planning mechanisms used to generate structured refactoring strategies. The algorithms, decision models, and planning architecture developed during this research can be protected through software copyright or intellectual property registration. Additionally, any proprietary machine learning models and training datasets used for predicting refactoring suitability may provide further competitive advantages for a commercial implementation of the system.

7. BUDGET AND JUSTIFICATION

The proposed research primarily requires **cloud computing resources** to execute experiments, store datasets, and deploy the prototype system. Most of the software tools used in this project, including Python libraries, SonarQube Community Edition, and other supporting frameworks, are **open-source and freely available**. Therefore, the overall budget mainly focuses on **cloud infrastructure and hosting services**, ensuring a cost-effective implementation.

Table 3: Estimated Budget for System Implementation and Experimentation

Table 5 Estimated Budget for System Implementation and Experimentation

Resource	Purpose	Estimated Cost (LKR)
Cloud Compute Instance (AWS EC2 / DigitalOcean Droplet)	Running code analysis agents and refactoring planning experiments	7,798.75 / month
Cloud Storage (AWS S3 / Google Cloud Storage)	Storing datasets, repositories, and generated refactoring plans	935.85 / month
Small Database Service (AWS RDS / Managed PostgreSQL)	Storing software metrics, code quality reports, and experiment results	3,119.50 / month
Web Hosting / Server Hosting	Hosting the prototype system interface and backend APIs	3,119.50 / month
Domain and Backup Storage	Domain registration and backup of experimental data	623.90 / month

Total Estimated Budget

Total Cost: 15,597.50 LKR per month

Justification of Expenses

Cloud Compute Instance

A cloud-based compute instance is essential to run the **multi-agent refactoring system**, including code analysis, planning, and validation processes. It provides sufficient

computational power to execute experiments on real-world software repositories efficiently and supports scalable processing when handling multiple workflows.

Cloud Storage

Cloud storage services are required to store:

- software repositories used in experiments,
- datasets and intermediate outputs,
- generated refactoring plans and logs.

This ensures **reliable, scalable, and secure data storage**, enabling easy access and management throughout the research lifecycle.

Database Service

A managed database service (e.g., PostgreSQL) is used to store structured data such as:

- software quality metrics,
- dependency relationships,
- refactoring plans and results,
- workflow logs and audit records.

This supports **efficient data retrieval, consistency, and traceability** of experimental results.

Web Hosting / Server Hosting

A hosting service is required to deploy the **prototype system**, including:

- backend APIs (Flask-based services),
- frontend dashboard (React-based interface).

This allows users (developers or evaluators) to interact with the system, submit repositories, review refactoring plans, and visualize results in a **realistic environment**.

Domain and Backup Storage

A small budget is allocated for:

- domain registration to provide a **public access point** for the system,
- backup storage to ensure **data reliability and recovery** in case of system failures.

This enhances the **availability, stability, and credibility** of the prototype system.

8. WORK BREAKDOWN STRUCTURE (WBS)

The project work is divided among the four team members according to the specialized agents in the multi-agent architecture. Each member is responsible for the design, implementation, and evaluation of a specific agent while collaborating on common tasks such as system integration, testing, and documentation.

Table 6 Task Distribution Among Team Members

Team Member	Component / Agent	Key Responsibilities
Member 1	Code Understanding Agent	Analyzing the source code structure, identifying code smells, generating complexity metrics, and producing structured quality reports.
Member 2	Refactoring Planning Agent	Generating intelligent refactoring strategies based on analysis results, prioritizing refactoring actions, and preparing machine-executable refactoring plans.
Member 3	Safe Code Transformation & Validation Agent	Applying AST-based code transformations, validating syntactic and semantic correctness, performing behavioral validation, and handling rollback when necessary.
Member 4	Developer Interaction & Workflow Orchestration Agent	Managing the interaction between developers and the system, coordinating workflow among agents, presenting refactoring plans and results through a dashboard or IDE interface.

Although each member is responsible for a specific component, several tasks are performed collaboratively to ensure system consistency and integration. These shared tasks include system architecture design, integration of all agents into a unified platform, testing and validation of the complete system, evaluation of experimental results, and preparation of research documentation.

The major development activities of the project are organized into phases that align with the research timeline. The project begins with literature review and system design activities, followed by implementation of the individual agents. After the development

stage, the agents are integrated into a unified system and evaluated using selected legacy code repositories. Finally, documentation and final reporting activities are completed.

Table 7 Project Timeline Alignment

Project Phase	Activities	Timeline
Phase 1	Literature review, problem analysis, and research proposal preparation	Month 1 – Month 2
Phase 2	System architecture design and agent specification	Month 3
Phase 3	Implementation of Code Understanding Agent	Month 4 – Month 5
Phase 4	Implementation of Refactoring Planning Agent	Month 4 – Month 5
Phase 5	Implementation of Safe Transformation & Validation Agent	Month 5 – Month 6
Phase 6	Implementation of Developer Interaction & Workflow Orchestration Agent	Month 5 – Month 6
Phase 7	Integration of agents and system testing	Month 7
Phase 8	Experimental evaluation and performance analysis	Month 8
Phase 9	Final documentation, report writing, and presentation preparation	Month 9

The workload distribution among team members is carefully balanced so that each member contributes equally to the development of the proposed system. Each agent requires comparable levels of research, design, and implementation effort. In addition, shared activities such as system integration, testing, evaluation, and documentation are carried out collaboratively by all members. This balanced distribution of tasks ensures efficient progress of the project while allowing each team member to develop expertise in a specific component of the multi-agent architecture.

9. GANTT CHART

The project is organized into multiple phases including literature review, system design, implementation of individual agents, system integration, evaluation, and final documentation. The schedule ensures that each stage of the project is completed within the academic timeline while allowing sufficient time for development, testing, and analysis.

Table 8- Gantt Chart

Project Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6	Month 7	Month 8	Month 9
Literature Review	●	●							
Problem Identification	●	●							
System Architecture Design		●	●						
Research Proposal Preparation	●	●							
Development of Code Understanding Agent			●	●					
Development of Refactoring Planning Agent			●	●					
Development of Transformation & Validation Agent				●	●				
Development of Workflow Orchestration Agent				●	●				
System Integration					●	●			
Testing and Validation						●	●		
Experimental Evaluation							●	●	
Final Report Writing							●	●	●
Final Presentation Preparation								●	●

10.REFERENCES LIST

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.
- [2] N. R. Jennings, “An agent-based approach for building complex software systems,” *Communications of the ACM*, vol. 44, no. 4, pp. 35–41, Apr. 2001.
- [3] C. Parnin and S. Rugaber, “Resumption strategies for interrupted programming tasks,” *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, Mar. 2011.
- [4] T. D. LaToza and B. A. Myers, “Developers ask reachability questions,” in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, 2010, pp. 185–194.
- [5] M. Ernst, J. Cockrell, W. Griswold, and D. Notkin, “Dynamically discovering likely program invariants to support program evolution,” *IEEE Transactions on Software Engineering*, vol. 27, no. 2, pp. 99–123, Feb. 2001.
- [6] T. Zimmermann, N. Nagappan, H. Gall, E. Giger, and B. Murphy, “Cross-project defect prediction,” in *Proc. ACM SIGSOFT Symposium on Foundations of Software Engineering*, 2009, pp. 91–100.

11.APPENDICES

The appendices provide supplementary materials that support the design, implementation, and evaluation of the proposed system. These materials offer additional detail that is not included in the main body of the report but is essential for completeness, transparency, and reproducibility of the research.

Appendix A: Survey Instruments

This appendix includes the questionnaires used to evaluate the usability and effectiveness of the system. The survey is designed using Likert-scale and open-ended questions to measure:

- ease of use of the developer dashboard,
- clarity of refactoring plans and workflow stages,
- developer trust in automated decisions,
- satisfaction with validation and rollback features.

The survey responses will be used to assess user experience and overall system acceptance.

Appendix B: Detailed System Diagrams

This section provides detailed diagrams illustrating the system architecture and workflow, including:

- overall multi-agent system architecture,
- interaction flow between agents,
- workflow orchestration process,
- sequence diagrams for key operations (e.g., approval, rollback).

These diagrams help in understanding the internal structure and communication mechanisms of the system.

Appendix C: Preliminary UI Mockups

This appendix includes initial design mockups of the developer dashboard, such as:

- repository submission interface,
- code smell visualization panel,
- refactoring plan review interface,
- before-and-after comparison view,
- rollback and validation controls.

These mockups demonstrate the intended user interaction and layout of the system interface.

Appendix D: Risk Analysis

This section identifies potential risks associated with the project and outlines mitigation strategies.

Key Risks:

- incorrect or unsafe automated refactoring,
- integration issues between multiple agents,
- performance limitations when handling large codebases,
- low user trust in automated decisions,
- system failures during transformation or validation stages.

Mitigation Strategies:

- implement approval-gated workflow,
- use version control for rollback support,
- perform thorough validation and testing,
- ensure transparency through audit logs and explanations,
- conduct usability evaluation to improve user trust.

Appendix E: Ethical Documents

This appendix includes documents related to ethical compliance in the research process:

- participant information sheet (if user studies are conducted),
- consent forms for usability evaluation participants,
- data handling and privacy guidelines,
- confirmation that only publicly available repositories are used.

These documents ensure that the research adheres to ethical standards and protects participant data and privacy.

